

CNN ② 성능을 개선하는 실험

CIFAR10에서 증강과 정규화의 역할을 구분한다.

Residual block을 구현하고 공정한 비교 실험을 한다.

오분류·클래스별 성능·일반화 간격으로 결과를 해석한다.

BASELINE
→ **EVIDENCE**

00-02분 · 이번 주에 만들 결과물 확인

90분 수업 지도

- 01 00-05 도입 · 05-20 개념 · 20-35 수식과 손계산
- 02 35-50 구현 설계 · 50-75 실습 · 75-85 분석
- 03 85-90 회고 · 부록과 자율 과제는 수업 이후

90 MIN / 개념 45 + 실습 25 + 분석·회고 20

02-05분 · 실습 노트북 열기

FashionMNIST에서 CIFAR10으로

- 01 흑백 28×28에서 컬러 32×32로 바뀐다.
- 02 배경·시점·색상 변화가 커진다.
- 03 같은 10개 클래스라도 분류 난도는 다르다.

$[B, 1, 28, 28] \rightarrow [B, 3, 32, 32]$

05-20분

실험의 출발점은 baseline

- 01 단순한 기준 모델을 먼저 고정한다.
- 02 변경할 요인은 한 번에 하나를 우선한다.
- 03 데이터 분할과 평가 절차를 모든 조건에서 공유한다.

가설: 좌우 반전·crop이 일반화에 도움을 줄까?

05-20분

증강은 학습 데이터에만

train

무작위 증강

정규화

모델

- 01 RandomCrop은 위치 변화에 대응하도록 한다.
- 02 HorizontalFlip은 좌우 변화에 대응하도록 한다.
- 03 validation/test는 고정된 전처리를 사용한다.

05-20분

정규화와 규제는 다르다

- 01 입력 Normalize는 데이터의 수치 범위를 조정한다.
- 02 BatchNorm은 중간 활성화의 통계를 사용한다.
- 03 weight decay·Dropout은 과적합 완화를 위한 수단이다.

입력 처리 / 활성화 처리 / 파라미터·표현 규제

05-20분

BatchNorm의 두 가지 모드

- 01 학습 시에는 미니배치 통계를 사용한다.
- 02 평가 시에는 누적된 running 통계를 사용한다.
- 03 작은 배치나 분포 변화에서는 성능이 흔들릴 수 있다.

Conv → BatchNorm → ReLU

05-20분

더 깊으면 항상 더 좋을까

- 01 깊이는 표현력을 늘리지만 최적화를 어렵게 만들 수 있다.
- 02 학습 데이터가 적으면 과적합도 커질 수 있다.
- 03 연결 구조와 학습 예산을 함께 고려해야 한다.

모델 크기 × 데이터 × 최적화 × 예산

05-20분

잔차 연결의 핵심

x

Conv-BN-ReLU

Conv-BN

$F(x)+x$

ReLU

- 01 블록은 x 전체 대신 수정량 $F(x)$ 를 학습한다.
- 02 입력 x 가 덧셈 경로로 전달된다.
- 03 덧셈하려면 두 경로의 shape가 같아야 한다.

05-20분

입력 Normalize 손계산

- 01 ToTensor 뒤 픽셀은 보통 0
- 02 1 범위다.
- 03 mean=0.5, std=0.5이면 0은 -1, 1은 1이다.
- 04 시각화에는 역변환을 적용한다.

$$\mathbf{x_norm} = (\mathbf{x}-0.5)/0.5; \mathbf{x} = 0.5 \mathbf{x_norm} + 0.5$$

20-35분

BatchNorm의 식

- 01 채널별 평균과 분산으로 활성화를 표준화한다.
- 02 학습 가능한 γ 와 β 가 스케일과 이동을 조정한다.
- 03 ϵ 는 분모가 0에 가까울 때 수치 불안정을 줄인다.

$$y = \gamma (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$

20-35분

Residual의 기울기 경로

- 01 $y=x+F(x)$ 이면 직접 경로의 미분이 1을 포함한다.
- 02 F 가 작아도 입력이 전달될 수 있다.
- 03 모든 gradient 문제가 자동으로 사라지는 것은 아니다.

$$dy/dx = I + dF/dx$$

20-35분

projection shortcut

- 01 stride 2나 채널 증가로 shape가 달라질 수 있다.
- 02 1×1 Conv와 같은 stride로 shortcut도 맞춘다.
- 03 덧셈은 브로드캐스팅에 기대지 않고 동일 shape로 한다.

F(x): [B,64,16,16] / P(x): [B,64,16,16]

20-35분

Dropout과 weight decay

- 01 Dropout은 학습 시 일부 활성화를 무작위로 비운다.
- 02 AdamW는 파라미터 감소를 optimizer update와 분리한다.
- 03 각 기법의 강도는 validation으로 선택한다.

p_dropout ↑ / weight_decay ↑ → 항상 성능 ↑ 는 아님

20-35분

학습률 곡선 읽기

- 01 너무 큰 lr은 손실을 진동·발산시킬 수 있다.
- 02 너무 작은 lr은 같은 예산에서 충분히 학습하지 못한다.
- 03 scheduler를 썼다면 실제 lr 기록을 함께 남긴다.

학습률 × 업데이트 수를 함께 비교한다

20-35분

Dataset 공유 함정 피하기

고정 인덱스

train Dataset / val
Dataset

Subset

DataLoader

- 01 분할 인덱스를 먼저 한 번만 만든다.
- 02 train용 Dataset에는 증강 transform을 준다.
- 03 validation용 Dataset에는 고정 transform을 준다.

35-50분

작은 CIFAR 분류기

$32 \times 32 \times 3$

$16 \times 16 \times 32$

$8 \times 8 \times 64$

GAP

10

- 01 Conv-BN-ReLU 두 블록으로 특징을 추출한다.
- 02 AdaptiveAvgPool로 공간 평균을 구한다.
- 03 Linear는 채널 수에서 클래스 수로 연결한다.

35-50분

최고 모델을 제대로 복사

validation loss가 개선될 때 state_dict를 복사한다.

참조만 저장하면 뒤의 학습으로 값이 바뀔 수 있다.

학습 후 최고 가중치를 복원해 분석한다.

```
best = copy.deepcopy(model.state_dict())  
model.load_state_dict(best)
```

35-50분

비교 조건을 표로 고정

- 01 baseline과 augmentation 모델은 같은 초기 seed를 쓴다.
- 02 분할, batch, lr, epoch와 평가 표본을 맞춘다.
- 03 벽시계 시간과 파라미터 수를 같이 기록한다.

비교: base / augment / residual(확장)

35-50분

실패 시 복구 경로

- 01 다운로드는 수업 전에 끝낸다.
- 02 CPU에서는 TRAIN_N과 EPOCHS를 줄인다.
- 03 학습이 길면 첫 비교까지만 하고 분석 셀로 이동한다.

먼저 끝까지 실행 → 그다음 예산 확대

35-50분

LAB 1 · 증강을 눈으로 검증

- 01 practice §1–3: 같은 이미지의 증강 8장을 그린다.
- 02 validation 이미지가 반복 호출에도 같은지 검사한다.
- 03 작은 모델과 residual block의 shape를 확인한다.

50–57분 / 산출물: 증강 격자 + shape assert

50–75분 · practice.ipynb와 함께 진행

LAB 2 · 두 조건 비교

- 01 practice §4: base와 augment를 같은 예산으로 학습한다.
- 02 validation 손실로 각 조건의 최고 모델을 저장한다.
- 03 곡선과 파라미터·시간을 표에 정리한다.

57-69분 / 산출물: 조건별 곡선과 결과 표

50-75분 · practice.ipynb와 함께 진행

LAB 3 · 선택과 최종 평가

- 01 practice §5–6: validation으로 최종 조건을 선택한다.
- 02 confusion matrix와 오분류 이미지를 확인한다.
- 03 선택한 모델만 test에서 한 번 평가한다.

69–75분 / 산출물: 최종 평가 + 실패 유형

50–75분 · practice.ipynb와 함께 진행

증강이 손해처럼 보일 때

- 01 같은 epoch에서 학습 문제가 더 어려워진다.
- 02 훈련 표본이나 업데이트 수가 부족할 수 있다.
- 03 변환이 과하거나 클래스 의미를 훼손했을 수 있다.

한 번의 하락 ≠ 증강은 쓸모없다

75-85분

ablation을 읽는 법

- 01 하나의 구성 요소를 제거하거나 추가한다.
- 02 나머지 조건을 고정해 차이를 해석한다.
- 03 상호작용이 있으면 한 기법의 효과가 조건에 따라 달라진다.

기준 → +증강 → +잔차 → 반복 seed

75-85분

좋은 결과 보고서

- 01 주장: 어떤 조건이 어떤 지표에서 달랐는가.
- 02 증거: 표·곡선·실패 예시를 제시한다.
- 03 한계: 예산·표본·seed·데이터 범위를 적는다.

주장 / 증거 / 한계 / 다음 실험

75-85분

출구 퀴즈

- 01 Q1. validation에도 RandomCrop을 넣어도 될까?
- 02 Q2. eval()은 gradient 계산을 자동으로 끝까?
- 03 Q3. 채널 수가 다르면 residual을 어떻게 더할까?

답보다 이유가 중요하다

85-90분

이미지에서 문장으로

- 01 CNN은 지역 패턴을 공유 가중치로 찾는다.
- 02 문장에서는 먼 토큰의 관계도 직접 연결하고 싶다.
- 03 다음 주에는 관계의 가중치를 입력마다 계산한다.

다음: Q·K·V를 직접 구현한다

85-90분

부록 · 전이 학습의 절차

- 01 사전학습 가중치와 해당 전처리를 함께 확인한다.
- 02 분류 헤드를 교체하고 먼저 헤드만 학습할 수 있다.
- 03 이후 작은 lr로 일부 또는 전체를 미세조정한다.

pretrained backbone → new head → fine-tune

수업 외 심화

부록 · freeze와 eval

- 01 requires_grad=False는 파라미터 gradient를 막는다.
- 02 eval은 BatchNorm·Dropout의 모드를 바꾼다.
- 03 동결된 backbone의 통계 갱신 여부도 결정해야 한다.

freeze ≠ eval

수업 외 심화

부록 · label smoothing

- 01 정답 확률 목표를 한 점에서 조금 분산한다.
- 02 과도한 확신을 완화할 수 있다.
- 03 강도가 크면 학습 신호를 약하게 만들 수 있다.

```
nn.CrossEntropyLoss(label_smoothing=0.1)
```

수업 외 심화

부록 · class imbalance

- 01 전체 정확도가 다수 클래스에 좌우될 수 있다.
- 02 클래스별 recall과 macro 평균을 함께 본다.
- 03 샘플링·가중 손실은 분포와 목적에 따라 선택한다.

macro recall = 클래스별 recall의 평균

수업 외 심화

부록 · mixed precision

- 01 일부 연산을 낮은 정밀도로 수행한다.
- 02 GPU에서 메모리와 속도 이점을 얻을 수 있다.
- 03 수치 안정성과 지원 dtype을 확인해야 한다.

속도 향상은 하드웨어와 모델에 의존한다

수업 외 심화

부록 · 재현성의 경계

- 01 seed는 난수 흐름을 고정하는 출발점이다.
- 02 device와 라이브러리 버전에 따라 결과가 달라질 수 있다.
- 03 완전한 bitwise 재현과 통계적 재현을 구분한다.

동일 설정 + 여러 seed + 분포 보고

수업 외 심화

부록 · 다음 실험 제안

- 01 residual on/off를 같은 조건으로 비교한다.
- 02 class recall을 구현하고 최악 클래스를 찾는다.
- 03 실패 사례를 한 가지 증강 가설로 연결한다.

자율 과제: residual ablation + 보고서

수업 외 심화

시각 보충 · 패치와 필터의 곱

입력 패치

1	2
3	4

×

공유 필터

1	0
0	-1

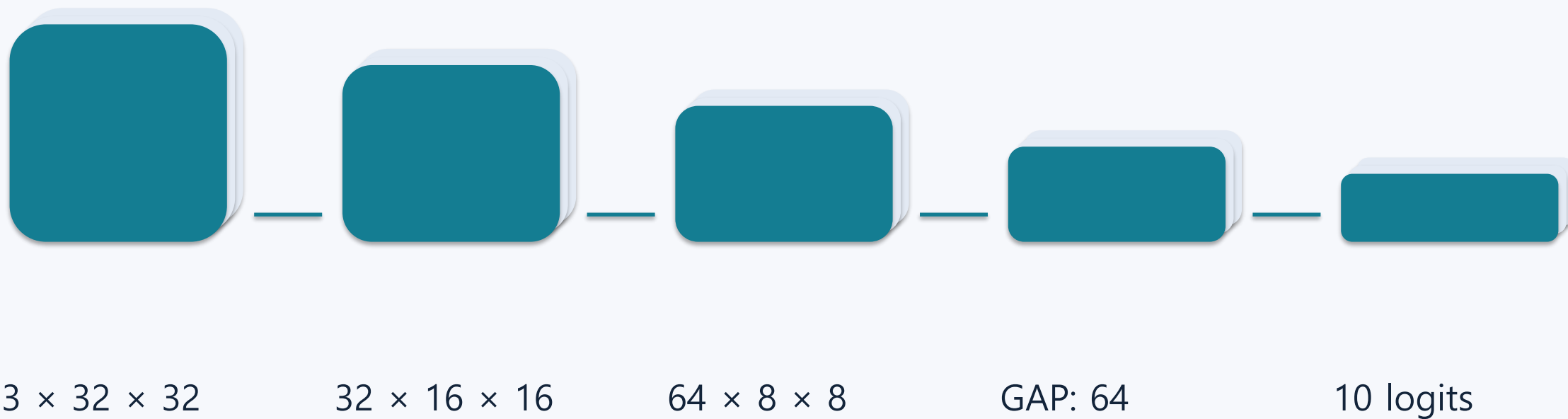
=

$$1 \times 1 + 2 \times 0 + 3 \times 0 + 4 \times (-1) = -3$$

bias=1을 더하면 -2 / 같은 필터를 다음 패치로 이동한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 해상도와 채널의 흐름



B는 모든 단계에서 유지된다. 채널 수와 공간 해상도를 서로 바꾸어 읽지 않는다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 혼동 행렬 읽기

설명용 3-class confusion matrix

	예측 A	B	C
정답 A	8	1	1
B	2	6	2
C	0	3	7

$$B \text{ recall} = 6 / (2+6+2) = 0.60$$

$$B \text{ precision} = 6 / (1+6+3) = 0.60$$

대각선: 정답

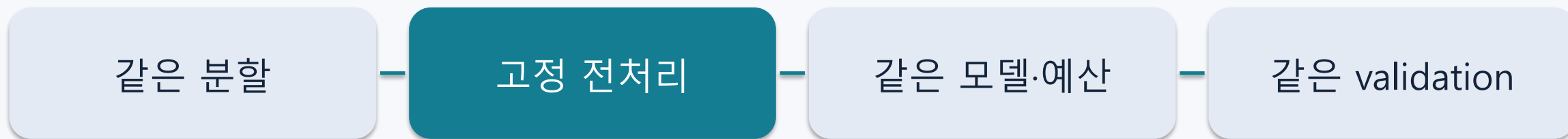
행 합: 실제 클래스 표본 수

열 합: 그 클래스로 예측한 수

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 공정한 실험의 두 경로

A · 기준



B · 변경



기법의 이름보다 무엇을 고정했고 무엇을 바꿨는지 먼저 기록한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

읽을거리와 출처

- 01 PyTorch · 이미지 분류 튜토리얼
- 02 Conv2d · 공식 API
- 03 Deep Residual Learning · He et al.

원문 링크는 발표자 노트와 주차별 README에 수록

생각하기 → 손으로 계산하기 → 코드로 검증하기